

Générateur de Pouvoir — Description du projet

Table of Contents

1. Vue d'ensemble	1
2. Cible technique	2
3. Concepts du domaine	3
3.1. Traits	3
3.2. Pouvoirs	3
3.3. Personnes	4
3.4. Espèces et genres	4
4. Hérité — Modèle de résilience	5
4.1. Principe	5
4.2. Combinaison entre parents pour un même trait	5
4.3. Transmission des traits inactifs	5
4.4. Cas spéciaux : mutation et enfant sans pouvoir	6
5. Cycle de vie et générations	7
5.1. Génération 0	7
5.2. Numéro de génération	7
5.3. Vieillesse	7
5.4. Mort	7
5.5. Création / édition manuelle d'individus	7
6. Modes d'utilisation	9
6.1. Reproduction manuelle	9
6.2. Simulation automatique	9
6.3. Sandbox	9
7. Interface utilisateur	10
7.1. Liste des individus	10
7.2. Fiche d'un individu	10
7.3. Arbre généalogique (page dédiée)	10
7.4. Autres pages	10
7.5. Modes d'affichage des traits d'un individu	11
8. Paramétrage	12
8.1. Paramètres de génération de pouvoir	12
8.2. Paramètres d'hérité	12
8.3. Paramètres de population	12
8.4. Paramètres d'espèce	12
8.5. Catalogues	13
9. Persistance	14
10. Points en suspens	15

1. Vue d'ensemble

Le projet est un **générateur de pouvoirs aléatoires** centré sur des **personnes** qui possèdent ces pouvoirs et les transmettent à leur descendance via un **système d'hérédité génétique**.

L'application doit permettre :

- de générer une population initiale de personnes pourvues (ou non) de pouvoirs aléatoires ;
- de faire reproduire ces personnes (manuellement ou en simulation automatique) sur plusieurs générations ;
- de visualiser et explorer l'arbre généalogique, les individus, leurs traits, leurs pouvoirs et leur génome ;
- d'expérimenter librement dans un **mode bac à sable** sans impacter la généalogie réelle ;
- d'exporter et importer toute la configuration et toutes les données générées.

2. Cible technique

Pour permettre à des utilisateurs sous **Windows, Mac, Linux**, ainsi que sur **smartphones (iOS et Android)** d'utiliser l'outil, le projet sera développé comme une **application web (PWA — Progressive Web App)** :

- accessible via navigateur, sans installation obligatoire ;
- installable sur le téléphone comme une app native (icône, plein écran) ;
- aucun backend obligatoire pour la V1 (tout tourne côté client) ;
- la persistance se fait exclusivement par **export / import de fichier** (pas de sauvegarde automatique en navigateur).

3. Concepts du domaine

3.1. Traits

Un **trait** est l'unité élémentaire qui compose un pouvoir. Il existe **6 types de traits** (types fixés en dur dans le programme ; le contenu de chaque type est éditable) :

Type de trait	Exemples
Remplacements	Pinces de crabe, Mandibule, Tentacules, Halo divin...
Parties du corps	Mains, Bras, Yeux, Peau, Cheveux...
Etats	Lumineux, Visqueux, Gazeux, Froid, Vibrant...
Elements	feu, eau, métaux, ombre, lumière, plume, électricité...
Ajouts	Fourrure, Plume, Tentacules, Carapace, Cristaux...
Actions	contrôle, créé, brule, téléporte, anime, soigne avec...

- Les **listes par défaut** sont celles du dossier `rsrc/ExempleTraits/`.
- Un utilisateur peut **ajouter ou retirer** des traits dans chaque catégorie à l'exécution.
- Un même mot peut apparaître dans plusieurs catégories (ex. *Fleur* dans Remplacements ET dans Ajouts) — chaque occurrence est un trait distinct, indépendant.

3.2. Pouvoirs

Un **pouvoir** est une combinaison de traits. Il existe **5 types de pouvoirs** (types fixés en dur dans le programme) :

Type de pouvoir	Composition de base (2 traits)	Exemple
Action sur Elements	Action + Element	"Contrôle feu"
Ajout sur Partie du corps	Ajout + Partie du corps	"Fourrure sur Bras"
Remplacement à la place de Partie du corps	Remplacement + Partie du corps	"Pinces de crabe à la place des Mains"
Partie du corps en Etat	Partie du corps + Etat	"Bras Lumineux"
Action sur Elements grâce à Partie du corps	Action + Element + Partie du corps	"Brûle eau grâce à Main"

Un pouvoir de base est composé du nombre minimal de traits indiqué ci-dessus. Si un individu hérite de traits actifs supplémentaires compatibles avec ce pouvoir, ceux-ci **enrichissent** le pouvoir existant. Exemple : un individu avec *Partie du corps : Bras*, *Etat : Lumineux* et *Etat : Vibrant* en actif aura un pouvoir "Bras Lumineux et Vibrant".



À spécifier ultérieurement : l'algorithme exact qui transforme la liste des traits

actifs d'un individu en une liste de pouvoirs lisibles. Cette logique est encore en réflexion et sera précisée plus tard.

3.3. Personnes

Une **personne** est décrite par les champs suivants :

Champ	Détail
id	Identifiant unique
nom	Généré aléatoirement (éditable)
sexe / genre	Enum paramétrable par espèce (cf. §3.4). Le genre " tout " est présent par défaut chez toutes les espèces.
espèce	Enum appartenant au catalogue des espèces (cf. §3.4). Défaut : " humain ".
génération	Entier. Vaut 0 pour la génération 0. Sinon = max(génération des parents) + 1.
âge	Entier. Naissance à un âge paramétrable par espèce.
vivant / décédé	Booléen. Si décédé : champ optionnel raison du décès (texte libre — par exemple "mort de vieillesse", "tué par l'utilisateur : <motif>"...).
parents	Liste de références vers d'autres personnes. null pour la génération 0.
enfants	Liste de références vers les enfants de cet individu. Mise à jour automatiquement à chaque naissance.
traits	Liste des traits hérités, chacun avec état (actif/inactif) et résilience (voir §4). Peut être null .
pouvoir(s)	Liste de pouvoirs dérivés des traits actifs. Peut être null .
notes	Texte libre éditable par l'utilisateur. Peut être null .

Un individu peut avoir **un nombre variable de pouvoirs**, qui émerge naturellement des combinaisons de traits actifs présents dans son génome.

3.4. Espèces et genres

Les **espèces** forment un catalogue éditable par l'utilisateur :

- l'utilisateur peut **ajouter, retirer et modifier** des espèces ;
- la seule espèce présente par défaut est "**humain**".

Pour chaque espèce, l'utilisateur définit **l'enum des genres / sexes** valides pour cette espèce. Quelle que soit l'espèce, un genre spécial "**tout**" est toujours présent par défaut :

- une personne de genre "**tout**" peut se reproduire **avec personne, ou avec un nombre arbitraire de partenaires de n'importe quel genre / sexe** (et de n'importe quelle espèce — cf. §6.1).

4. Hérité — Modèle de résilience

4.1. Principe

Lorsqu'un enfant naît, **il hérite de la totalité des traits de tous ses parents**. La question n'est donc pas *est-ce qu'il hérite du trait* mais *est-ce que le trait sera actif ou inactif chez lui*.

Chaque trait porté par un individu possède une **résilience** : un pourcentage qui représente sa probabilité d'être transmis **actif** à la descendance.

- Si un trait est tiré **actif** chez l'enfant → un **bonus** est appliqué à sa résilience chez l'enfant.
- Si un trait est tiré **inactif** chez l'enfant → un **malus** est appliqué à sa résilience chez l'enfant.
- Tous les traits ont une **résilience maximale** au-dessus de laquelle le bonus ne s'applique plus.
- Si la résilience tombe sous un **seuil de disparition** (par défaut 2 %), le trait disparaît du génome de l'enfant. **C'est la seule manière pour qu'un trait disparaisse définitivement de la lignée.**

4.2. Combinaison entre parents pour un même trait

Pour chaque parent qui porte le trait X (qu'il l'ait actif ou inactif), on effectue un **tirage individuel selon la résilience que ce parent a pour X** afin de déterminer si **ce parent transmet le trait actif ou inactif à l'enfant**. On agrège ensuite tous les tirages :

Cas 1 — Un seul parent porte le trait :

- Un seul tirage est effectué.
- Si le trait est transmis **actif** → trait actif chez l'enfant, résilience initiale = résilience du parent porteur, **bonus** appliqué.
- Si le trait est transmis **inactif** → trait inactif chez l'enfant, résilience initiale = résilience du parent porteur, **malus** appliqué.

Cas 2 — Plusieurs parents portent le trait :

On regarde le **résultat des tirages** (et non l'état actif / inactif chez les parents) :

- **Aucun tirage ne donne actif** (tous les parents porteurs ont transmis inactif) → trait inactif chez l'enfant, résilience initiale = **la plus haute** parmi les parents porteurs, **malus** appliqué.
- **Un seul tirage donne actif** → trait actif chez l'enfant, résilience initiale = résilience du parent dont le tirage est actif, **bonus** appliqué.
- **Plusieurs tirages donnent actif** → trait actif chez l'enfant, résilience initiale = **la plus haute** parmi les parents porteurs, le **bonus est appliqué autant de fois qu'il y a de tirages actifs** (ex. deux parents → 2× bonus ; trois parents → 3× bonus si tous les trois ont transmis actif).

4.3. Transmission des traits inactifs

Un trait **inactif** chez un individu **est quand même transmis** à ses enfants. La même logique

s'applique : tirage actif/inactif selon la résilience, bonus si actif, malus si inactif. Un génome peut donc "se réveiller" plusieurs générations plus tard.

4.4. Cas spéciaux : mutation et enfant sans pouvoir

À chaque naissance, deux événements probabilistes peuvent se produire (chacun avec un taux paramétrable) :

Mutation :

- Tous les traits parentaux sont hérités **inactifs**.
- Un **nouveau pouvoir aléatoire** est tiré, exactement avec la même logique que pour la génération 0 (voir §5.1).
- Ses traits constitutifs sont actifs avec une résilience initiale paramétrable.

Naissance sans pouvoir :

- Tous les traits parentaux sont hérités **inactifs**.
- Aucun pouvoir n'est tiré pour l'enfant.
- Ses descendants peuvent quand même réactiver les traits.

Option globale : malus sur le génome en cas de mutation / enfant sans pouvoir

- Activable / désactivable.
- **Par défaut désactivée** : les traits inactifs sont hérités sans pénalité de résilience.
- Dans ces deux cas spéciaux, si plusieurs parents partagent un trait, on conserve simplement la **résilience la plus élevée**.

5. Cycle de vie et générations

5.1. Génération 0

L'utilisateur paramètre :

- le **nombre d'individus** à créer ;
- le **pourcentage de chance** que chaque individu possède un pouvoir aléatoire.

Pour chaque individu créé, le programme :

1. **tire d'abord** si l'individu **possède un pouvoir ou non**, selon le pourcentage donné par l'utilisateur ;
2. **uniquement si oui** : tire un **type de pouvoir** parmi les 5, pondéré ;
3. tire les **traits nécessaires** à former ce pouvoir (selon les pondérations des traits) ;
4. attribue à chaque trait une **résilience initiale** paramétrable.

Si le tirage de l'étape 1 dit non, l'individu naît sans pouvoir ni trait.

5.2. Numéro de génération

Le **numéro de génération** d'un enfant est $\max(\text{numéro de génération des parents}) + 1$.

5.3. Vieillesse

- À chaque reproduction (manuelle entre individus précis ou en simulation), les **parents et tous leurs ancêtres encore vivants** vieillissent.
- L'âge gagné est tiré aléatoirement entre **âge_min** et **âge_max** paramétrés **par espèce**.
- Chaque espèce a une **durée de vie** paramétrable au-delà de laquelle l'individu meurt.

5.4. Mort

- Un individu mort reste dans l'arbre généalogique, marqué "décédé".
- Il ne peut plus se reproduire.
- L'utilisateur peut **tuer manuellement** un individu et **noter la raison de sa mort** (texte libre stocké dans le champ **raison du décès**).
- L'utilisateur peut activer un mode **immortalité** global qui désactive la mort naturelle.

5.5. Création / édition manuelle d'individus

L'utilisateur peut :

- créer un **nouvel individu personnalisé** en renseignant librement tous les champs (espèce,

sexe, traits, pouvoirs, notes...) ;

- créer un **nouvel individu basé sur un individu existant** (clonage éditable, pour gagner du temps).

6. Modes d'utilisation

6.1. Reproduction manuelle

L'utilisateur sélectionne **1, 2 ou plus** d'individus (peu importe leur genre ou leur espèce ; par défaut 2) et déclenche la reproduction. Le nombre d'enfants suit l'un des modes paramétrables :

- **fixe** : toujours N enfants par accouplement ;
- **aléatoire** : nombre tiré dans une plage [**min**, **max**].

6.2. Simulation automatique

L'utilisateur lance le passage à la génération suivante pour **tout ou partie de la population**. Le programme forme les couples (selon une stratégie configurable), produit les enfants, et fait vieillir les générations précédentes.

6.3. Sandbox

Un mode bac à sable qui permet de :

- créer des individus temporaires ;
- les faire reproduire entre eux ou avec des individus réels ;
- explorer des scénarios sans affecter la généalogie réelle ;
- valider l'ensemble via un bouton "**make it real**" qui injecte les modifications du bac à sable dans la généalogie principale.

7. Interface utilisateur

L'application propose plusieurs **pages** distinctes.

7.1. Liste des individus

Page qui liste **tous les individus** avec recherche et filtres (par génération, espèce, trait, pouvoir, statut vivant/décédé, etc.). Chaque ligne / carte affiche **uniquement les informations globales** :

- nom,
- génération,
- pouvoir(s).

Un clic sur un individu ouvre sa **fiche individu** (§7.2).

7.2. Fiche d'un individu

Page dédiée à un individu, qui contient :

- ses **informations globales** (nom, génération, pouvoir(s), espèce, sexe/genre, âge, statut vivant/décédé, notes, parents...) ;
- ses **traits**, affichés selon le mode d'affichage actif (cf. §7.5) ;
- un **arbre généalogique** centré sur lui, avec une **profondeur N sélectionnable** (par défaut 2) : N générations d'ancêtres au-dessus et N générations de descendants au-dessous, dans la mesure du possible.

Chaque case de l'arbre contient **nom, génération et pouvoir(s)** de l'individu correspondant. **Un clic sur une case de l'arbre** ouvre la fiche de l'individu cliqué (la page se recentre sur lui).

7.3. Arbre généalogique (page dédiée)

Page qui affiche **le même arbre** que celui de la fiche individu (§7.2), centré sur le même individu et avec la même profondeur N sélectionnable, mais **sans les informations latérales** : on n'a que l'arbre, en grand, pour navigation visuelle.

Les cases contiennent les mêmes infos (nom, génération, pouvoir(s)), et un clic sur une case bascule la page sur l'arbre de cet individu.

7.4. Autres pages

- **Écran de reproduction manuelle** : sélection des parents (1, 2 ou plus, cf. §6.1) et déclenchement.
- **Écran de simulation automatique** : lancement de N générations automatiques, suivi d'évolution / statistiques.
- **Écran sandbox** : voir §6.3.

7.5. Modes d’affichage des traits d’un individu

Trois modes au choix, applicables aux pages qui affichent les traits d’un individu (fiche individu notamment) :

- **Mode 1** : pouvoirs uniquement ;
- **Mode 2** : pouvoirs + traits actifs ;
- **Mode 3** : pouvoirs + traits actifs + traits inactifs + résilience de chaque trait.

8. Paramétrage

Tout le système doit être paramétrable. Les paramètres se déclinent à trois niveaux pour la plupart : global, par type de trait, par trait individuel.

8.1. Paramètres de génération de pouvoir

- Poids de chaque **type de pouvoir** dans le tirage.
- Poids de chaque **type de trait** dans les tirages.
- Poids de chaque **trait individuel** dans les tirages.
- Résilience initiale attribuée aux traits d'un pouvoir nouvellement tiré.

8.2. Paramètres d'hérédité

- Résilience initiale : globale, par type de trait, par trait.
- Résilience maximale : globale, par type de trait, par trait.
- Facteur de **bonus** appliqué quand un trait est tiré actif.
- Facteur de **malus** appliqué quand un trait est tiré inactif.
- **Seuil de disparition** (par défaut 2 %).
- Taux de **mutation** par naissance.
- Taux d'**enfant sans pouvoir** par naissance.
- Option **activer/désactiver le malus** sur le génome en cas de mutation et/ou d'enfant sans pouvoir (désactivé par défaut).

8.3. Paramètres de population

- Nombre d'individus en génération 0.
- Pourcentage de chance qu'un individu de génération 0 ait un pouvoir.
- Mode du nombre d'enfants par accouplement : fixe ou aléatoire, avec sa valeur ou ses bornes.

8.4. Paramètres d'espèce

Pour chaque espèce :

- Durée de vie.
- Âge à la naissance.
- Plage de vieillissement par tick [**âge_min**, **âge_max**].
- Mode immortalité (oui / non).

8.5. Catalogues

- Listes de traits éditables (ajout / suppression dans chaque type).
- Les listes par défaut sont celles de `rsrc/ExempleTraits/`.
- Les 6 types de traits et 5 types de pouvoirs sont **fixés** dans la V1 ; ils pourront être étendus dans des versions futures.

9. Persistance

- **Aucune sauvegarde automatique** côté navigateur.
- L'utilisateur peut **exporter** trois types de fichiers (JSON conseillé) :
 1. **Fichier de configuration seule** : tous les paramètres, catalogues de traits, catalogue d'espèces et leurs genres, pondérations, facteurs bonus/malus, seuils, etc.
 2. **Fichier de données générées seul** : tous les individus, la généalogie, l'historique.
 3. **Fichier combiné** : configuration + données dans un seul fichier.
- Chaque fichier contient un **identifiant de type** (par exemple un champ `kind: "config" \` | `"data" \` | `"full"` en tête du JSON) qui permet au programme, **au moment de l'import**, de reconnaître automatiquement de quel type de fichier il s'agit et d'appliquer le bon traitement (remplacer / fusionner la config seule, les données seules, ou les deux).
- Le même format permet de partager un état (entier ou partiel) entre appareils ou entre utilisateurs.

10. Points en suspens

À préciser plus tard dans le projet :

1. **Algorithme de construction des pouvoirs à partir des traits actifs** — règles exactes pour combiner les traits actifs d'un individu en un (ou plusieurs) pouvoirs lisibles, notamment quand plusieurs traits du même type sont actifs (cf. §3.2).
2. **Stratégie d'appariement** en simulation automatique (totalement aléatoire ? contraintes par espèce, génération, lien de parenté ?).
3. **Statistiques affichées** en mode simulation automatique.
4. **Format précis** du fichier d'export / import.
5. **Maquettes UI** des différents écrans.